

模式识别 实验报告

- 学号： 23071005
- 姓名： 侯景祺
- 日期： 2025 年 6 月 3 日

实验一：贝叶斯分类器 手写数字识别

一、方法描述

1.1 实验目标

研究 PCA 降维对高斯朴素贝叶斯 (Gaussian Naive Bayes, GNB) 和核密度朴素贝叶斯 (Kernel Density Naive Bayes, KNB) 分类器在 MNIST 手写数字识别任务上性能的影响。

比较在不同数量主成分下, GNB 和 KNB 分类器的分类准确率、训练时间及预测时间。

理解这两种贝叶斯分类器的基本原理, 以及在 PCA 处理后的数据上, 不同的概率密度估计方法 (高斯分布 vs. 核密度估计) 如何影响分类结果。

1.2 数据集

实验使用标准的 MNIST 数据集, 该数据集从 .mat 文件加载。

训练集: 包含 60,000 个样本。

测试集: 包含 10,000 个样本。

图像属性: 每个样本是 28×28 像素的灰度图像。

标签: 图像对应的数字标签, 范围从 0 到 9。

1.3 数据预处理

针对输入数据, 依次进行了以下预处理步骤:

1. 数据加载与转换:

从 .mat 文件中加载原始图像和标签数据。

将原始图像数据从 (height, width, num_samples) 的维度转置为 (num_samples, height, width) 的格式。

将转置后的二维图像数据展平为 (num_samples, 784) 的二维数组, 其中 784 (28×28) 代表每个样本的特征数量。

将标签数据转换为一维数组。

2. 归一化(Normalization):

将展平后的像素值从原始的 [0, 255] 范围归一化到 [0, 1] 区间, 通过将所有像素值除以 255.0 实现。

3. 标准化 (Standardization):

对归一化后的训练数据和测试数据应用 sklearn.preprocessing.StandardScaler 进行标准化处理, 使得每个特征 (像素点) 均值为 0, 标准差为 1。此步骤对于 PCA 的有效性至关重要。

4. 主成分分析(PCA) 降维:

对标准化后的训练数据应用 PCA 进行降维, 并用训练好的 PCA 模型转换测试数据。

实验探索了不同数量的主成分, 具体设置为: n_components = [10, 20, 30, 40, 50]。

1.4 分类器实现与概率密度估计方法的选择

本实验实现了并比较了以下两种朴素贝叶斯分类器:

1. 高斯朴素贝叶斯(Gaussian Naive Bayes, GNB):

实现: 使用 sklearn.naive_bayes.GaussianNB。

概率密度估计: 此模型假设经过 PCA 降维后的每一个主成分特征, 在每个类别下都服从高斯 (正态) 分布。这是一种参数化的概率密度估计方法。

2. 核密度朴素贝叶斯(Kernel Naive Bayes, KNB):

实现: 通过自定义的 KernelNaiveBayes 类实现, 该类内部使用 sklearn.neighbors.KernelDensity 进行概率密度估计。

概率密度估计: 此模型对每个类别下的每个主成分特征使用核密度估计 (一种非参数化方法) 来估计其概率分布。

核函数(Kernel): 选用高斯核 (kernel='gaussian')。

带宽(Bandwidth): 固定设置为 1.0。脚本中指出，带宽是影响 KDE 性能的关键参数，通常需要通过交叉验证等方法进行调优。

1.5 实验工具与环境

- **编程语言:** Python 3.11
- **主要库:**
 - scipy: 用于加载 .mat 格式的数据文件。
 - numpy: 用于高效的数值计算和数据处理。
 - scikit-learn: 用于实现朴素贝叶斯分类器模型 (GaussianNB, BernoulliNB, MultinomialNB) 及性能评估 (accuracy_score, classification_report)。
 - scikit-learn-intelex: 尝试用于通过 Intel® oneAPI Data Analytics Library (oneDAL) 对 Scikit-learn 算法进行加速 (主要针对 CPU 优化)。
- **运行环境:** Windows 系统，在一个 Conda 虚拟环境中执行。

1.6 评估指标

主要使用以下指标评估和比较模型性能：

- **准确率:** 在训练集和测试集上正确分类的样本数占总样本数的比例，使用 accuracy_score 计算。
- **训练时间:** 训练各个分类器模型(model.fit()) 所需的时间。
- **测试集预测时间:** 模型在测试集上进行预测(model.predict()) 所需的时间。
- **分类报告:** 包含每个数字类别 (0-9) 的精确率(Precision)、召回率 (Recall) 和 F1-score，使用 classification_report 生成。
- **混淆矩阵:** 为训练集和测试集分别生成混淆矩阵，以可视化分类结果，使用 plot_confusion_matrix_func 绘制。
- **PCA 相关指标:**
 - PCA 降维所需时间。
 - 所选主成分解释的总方差比例。

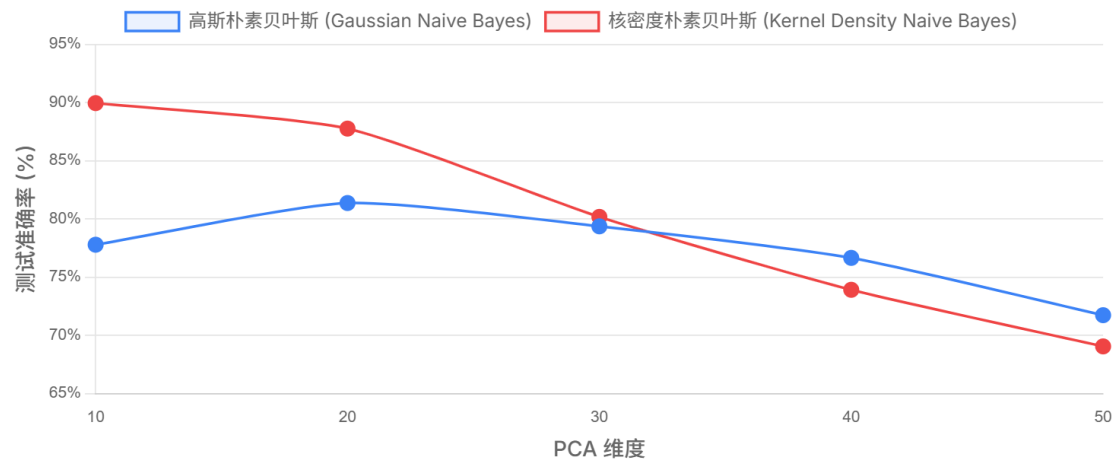
二、实验结果

本实验系统地评估了高斯朴素贝叶斯 (GNB) 和核密度朴素贝叶斯 (KNB) 分类器在经过 PCA 降维（主成分数分别设置为 10, 20, 30, 40, 50）后的性能。所有实验中，KNB 的带宽固定为 1.0。关键性能指标总结如下表所示：

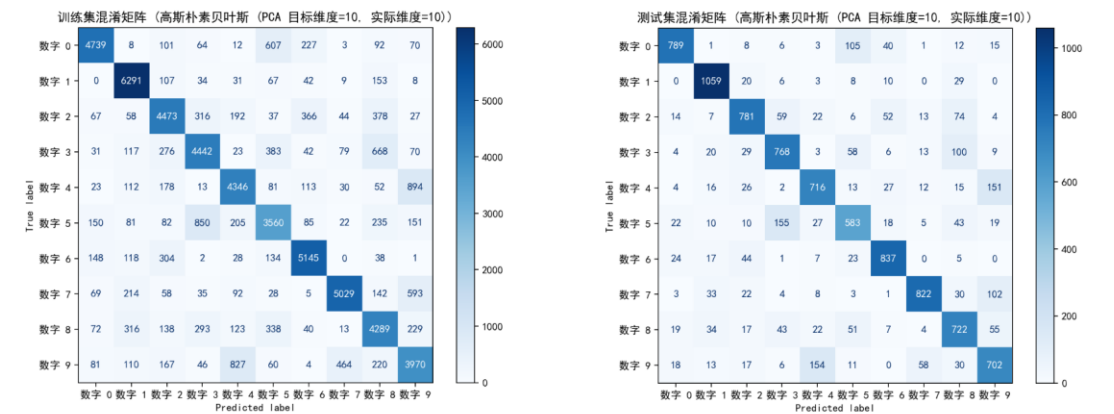
模型类型	PCA 设置 维度	实际 PCA 维度	解释的总方差 比例(%)	训练准确 率(%)	测试准确 率(%)	训练时 间(s)	测试预测时 间(s)	KDE 带宽
高斯朴素 贝叶斯	10	10	27.67	77.14	77.79	0.01	0.01	N/A
核密度朴 素贝叶斯	10	10	27.67	99.73	89.95	0.05	16.17	1
高斯朴素 贝叶斯	20	20	38.06	80.39	81.37	0.01	0.02	N/A
核密度朴 素贝叶斯	20	20	38.06	99.99	87.76	0.06	20.86	1
高斯朴素 贝叶斯	30	30	45.19	79.04	79.37	0.01	0.03	N/A

核密度朴素贝叶斯	30	30	45.19	100	80.18	0.1	22.86	1
高斯朴素贝叶斯	40	40	50.64	76.72	76.66	0.01	0.03	N/A
核密度朴素贝叶斯	40	40	50.64	100	73.92	0.12	31.35	1
高斯朴素贝叶斯	50	50	55.11	71.86	71.74	0.02	0.04	N/A
核密度朴素贝叶斯	50	50	55.11	100	69.06	0.17	33.86	1

测试准确率与PCA维度的关系

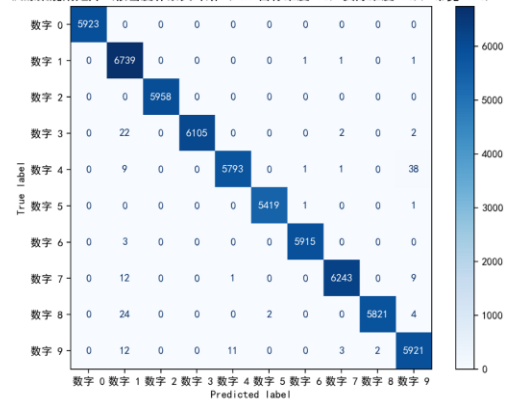


相关混淆矩阵如下：

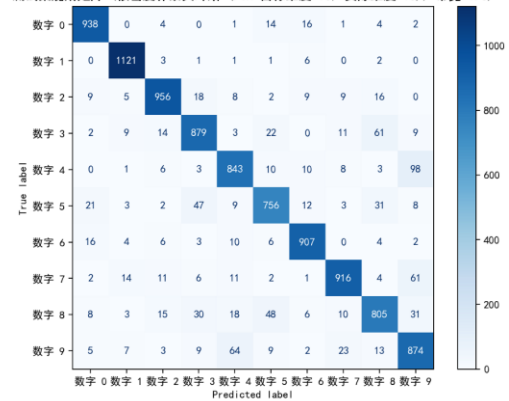


维度=10, GNB

训练集混淆矩阵 (核密度朴素贝叶斯 (PCA 目标维度=10, 实际维度=10), 带宽=1.0)

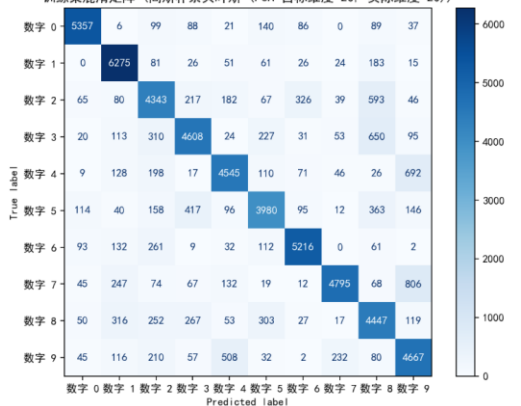


测试集混淆矩阵 (核密度朴素贝叶斯 (PCA 目标维度=10, 实际维度=10), 带宽=1.0)

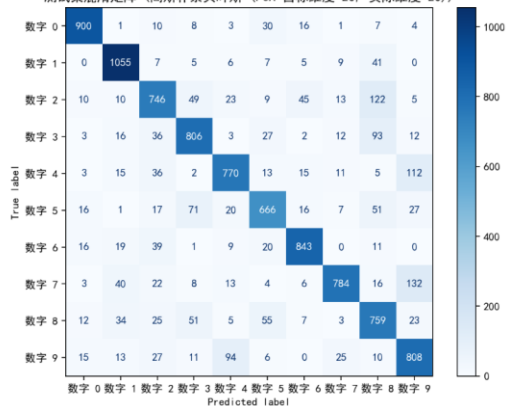


维度=10, KNB

训练集混淆矩阵 (高斯朴素贝叶斯 (PCA 目标维度=20, 实际维度=20))

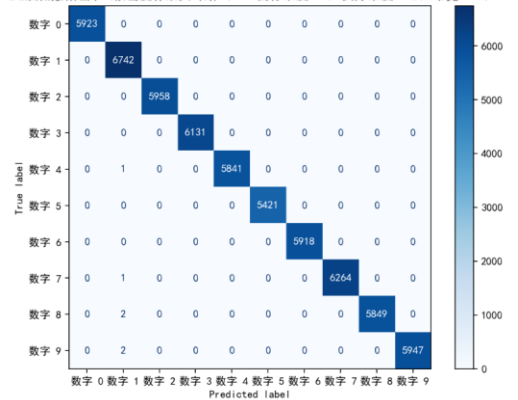


测试集混淆矩阵 (高斯朴素贝叶斯 (PCA 目标维度=20, 实际维度=20))

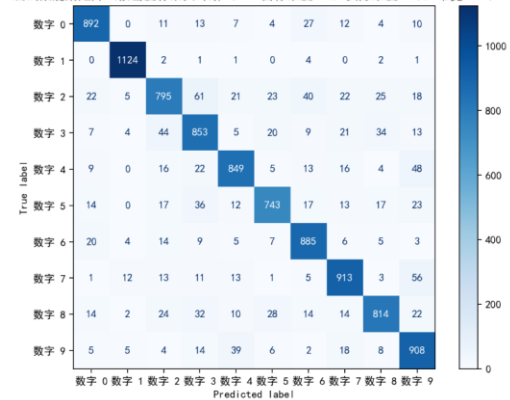


维度=20, GNB

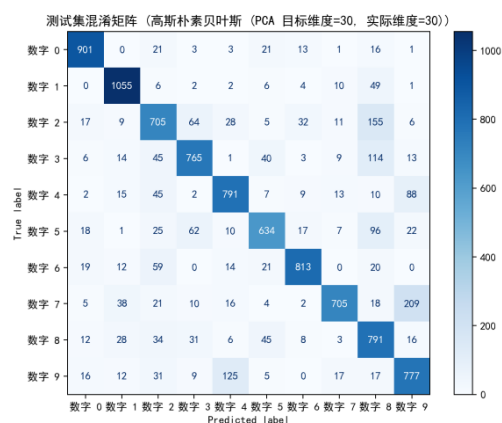
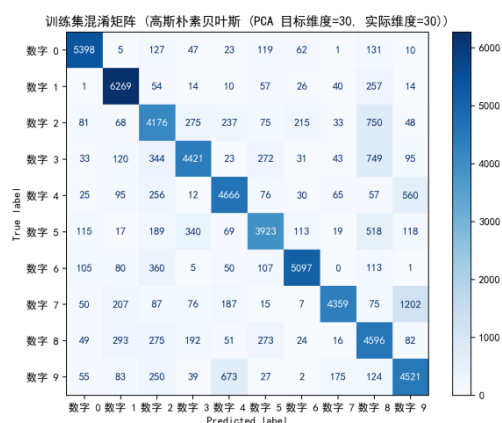
训练集混淆矩阵 (核密度朴素贝叶斯 (PCA 目标维度=20, 实际维度=20), 带宽=1.0)



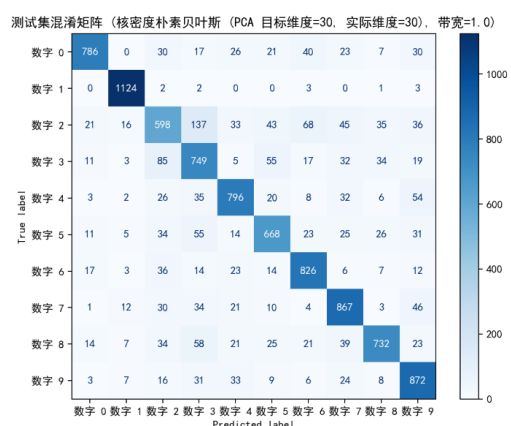
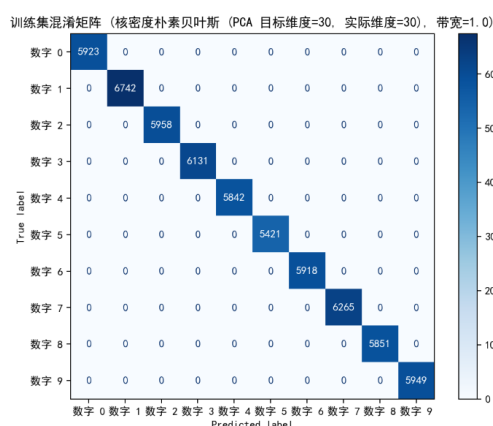
测试集混淆矩阵 (核密度朴素贝叶斯 (PCA 目标维度=20, 实际维度=20), 带宽=1.0)



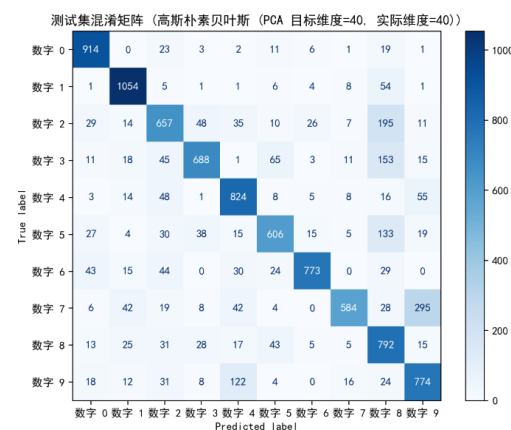
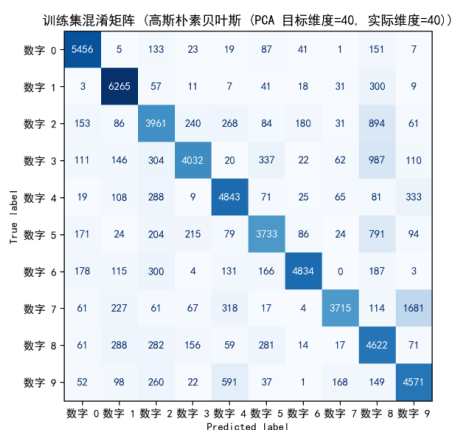
维度=20, KNB



维度=30, GNB

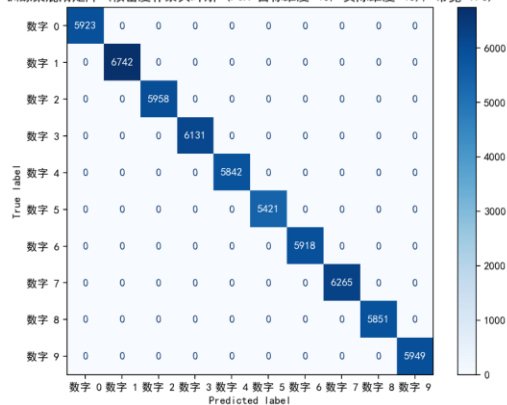


维度=30, KNB

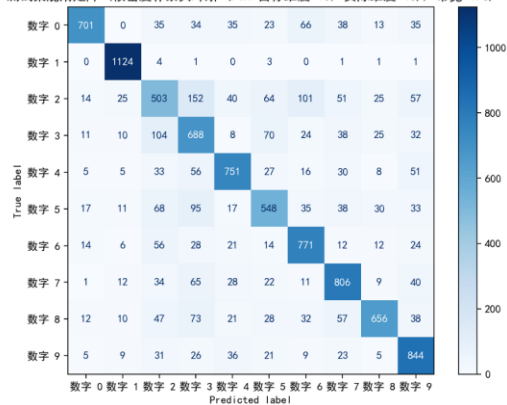


维度=40, GNB

训练集混淆矩阵 (核密度朴素贝叶斯 (PCA 目标维度=40, 实际维度=40), 带宽=1.0)

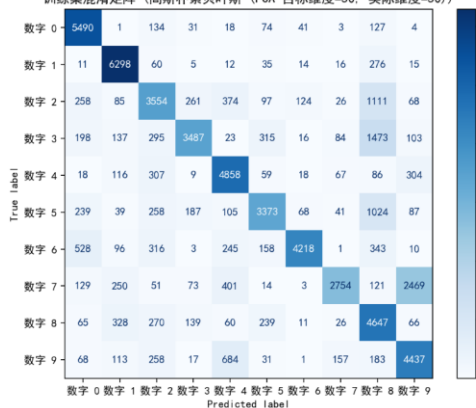


测试集混淆矩阵 (核密度朴素贝叶斯 (PCA 目标维度=40, 实际维度=40), 带宽=1.0)

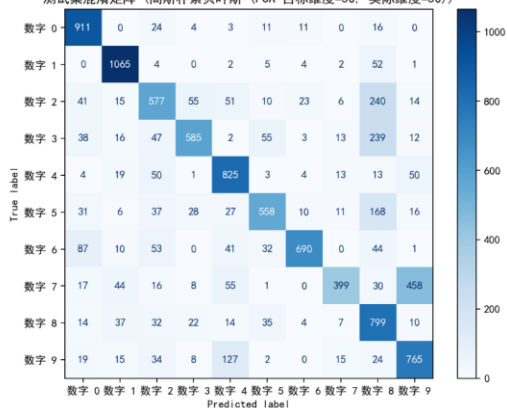


维度=40, KNB

训练集混淆矩阵 (高斯朴素贝叶斯 (PCA 目标维度=50, 实际维度=50))

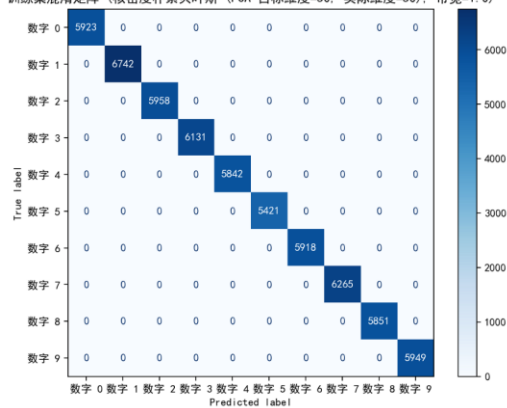


测试集混淆矩阵 (高斯朴素贝叶斯 (PCA 目标维度=50, 实际维度=50))

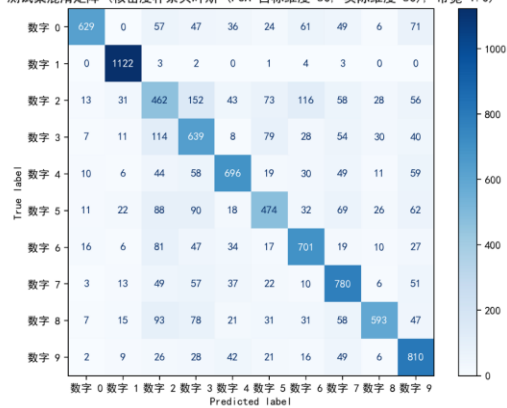


维度=50, GNB

训练集混淆矩阵 (核密度朴素贝叶斯 (PCA 目标维度=50, 实际维度=50), 带宽=1.0)



测试集混淆矩阵 (核密度朴素贝叶斯 (PCA 目标维度=50, 实际维度=50), 带宽=1.0)



维度=50, KNB

主要观察结果:

- 高斯朴素贝叶斯(GNB):

GNB 的测试准确率在 PCA 维度为 20 时达到峰值 (81.37%), 此时解释的总方差比例为 38.06%。

在 PCA 维度从 10 增加到 20 时, 准确率有所提升; 但当维度进一步增加到 30, 40, 50 时, 测试准确率反而呈现下降趋势。

训练时间和测试预测时间都非常短, 基本在 0.01-0.04 秒之间, 显示出极高的计算效

率。

训练准确率与测试准确率较为接近，未出现明显的过拟合。

- **核密度朴素贝叶斯(KNB):**

KNB 的测试准确率在 PCA 维度为 10 时达到峰值 (89.95%)，显著高于所有 GNB 的测试结果。此时解释的总方差比例仅为 27.67%。

随着 PCA 维度的增加 (从 10 到 50)，KNB 的测试准确率持续下降，从 89.95%降至 69.06%。

KNB 的训练准确率在所有 PCA 维度下都非常高 (接近或达到 100%)，但在测试集上的表现显示出随着维度增加，过拟合现象可能加剧，或者固定带宽 1.0 在高维空间中不再适用。

训练时间虽然也较短 (0.05-0.17 秒)，但测试预测时间远高于 GNB，从 PCA 维度 10 的约 16 秒增加到维度 50 的约 34 秒，显示其预测阶段计算成本较高且随维度增加而增加。

- **模型对比:**

在较低的 PCA 维度 (特别是 10 维) 下，KNB 的性能远优于 GNB。KNB 在 10 维 PCA 下获得了本次实验最高的测试准确率 89.95%。

GNB 的最佳性能出现在 20 维 PCA 下 (81.37%)。

随着维度的增加，KNB 的性能下降速度比 GNB 更快，且在 30 维之后，其准确率开始低于 GNB 在 20 维时的峰值，甚至在 40 维和 50 维时低于 GNB 在对应维度的表现。

计算效率方面，GNB 在训练和测试预测上都远快于 KNB。

- **混淆矩阵和分类报告分析 (基于每个 PCA 维度和模型的详细输出):**

GNB (PCA=20, Acc=81.37%): 分类报告显示，数字“0” (precision=0.92, recall=0.92, f1=0.92) 和数字“1” (precision=0.88, recall=0.93, f1=0.90) 以及数字“6” (precision=0.88, recall=0.88, f1=0.88) 识别效果较好。数字“8” (f1=0.73) 和数字“2” (f1=0.75) 的识别效果相对较差。

KNB (PCA=10, Acc=89.95%): 分类报告显示，各数字的识别都较为均衡且表现良好，例如数字“0” (f1=0.95)，数字“1” (f1=0.97)，数字“6” (f1=0.94)。数字“9” (f1=0.83) 和数字“8” (f1=0.84) 相对略低，但整体优于 GNB 的最佳情况。

随着 KNB 维度的增加，其分类报告中各个类别的 F1 分数也普遍下降，表明模型区分能力减弱。

三、模型评价

- **高斯朴素贝叶斯 with PCA:**

性能随 PCA 维度的变化: GNB 的测试准确率在 PCA 维度从 10 增加到 20 时，从 77.79%提升至 81.37%。然而，当维度进一步增加到 30、40 和 50 时，测试准确率反而下降，分别降至 79.37%、76.66%和 71.74%。这表明对于 GNB，并非越高的 PCA 维度就一定带来更好的性能，20 维似乎是一个较优的平衡点。

优点: GNB 的训练和预测速度极快，所有情况下的训练时间都在 0.01-0.02 秒左右，预测时间在 0.01-0.04 秒左右。这使其在对实时性要求高的场景下具有优势。

缺点: GNB 的整体准确率上限不高，最高仅达到 81.37%。这可能是由于 PCA 降维后的主成分特征的实际分布与高斯分布的假设仍有较大差距，且朴素贝叶斯模型忽略了特征间的相关性，即便 PCA 已尝试解耦特征。

计算成本: 计算成本非常低，训练和预测都极为迅速，对硬件资源要求低。

- **核密度朴素贝叶斯 with PCA:**

性能随 PCA 维度的变化: KNB 的测试准确率在 PCA 维度为 10 时达到最高值 89.95%。令人意外的是, 随着 PCA 维度的增加, 其测试准确率持续显著下降: 20 维时为 87.76%, 30 维为 80.18%, 40 维为 73.92%, 50 维时更是跌至 69.06%。这表明在本实验固定的带宽 (1.0) 下, KNB 对维度的增加非常敏感, 高维情况下性能急剧恶化, 可能存在“维度灾难”或固定带宽在高维空间中不再适用的问题。

带宽影响: 实验中带宽固定为 1.0。KNB 性能对带宽选择高度敏感。在 10 维时, 此带宽表现良好; 但在更高维度, 此固定带宽可能导致对数据分布的估计不准确 (可能过拟合或欠拟合), 从而影响分类性能。

优点: 在较低维度 (10 维) 下, KNB 展现了远超 GNB 的分类准确率 (89.95% vs 77.79%), 说明其非参数特性在能更好捕捉数据原始分布时具有优势。训练集准确率极高 (接近 100%) 也显示了其强大的拟合能力。

缺点:

计算成本高: 预测时间远高于 GNB, 且随维度增加而显著增长 (10 维时 16.17 秒, 50 维时 33.86 秒)。这使其在需要快速预测的场景下受限。

对维度和带宽敏感: 性能随维度增加急剧下降, 且高度依赖带宽参数的正确选择。固定带宽在高维下表现不佳。

过拟合风险: 极高的训练集准确率和随维度增加而下降的测试集准确率, 暗示模型在高维时可能出现了更严重的过拟合。

- **综合比较:**

准确率对比:

KNB 在 PCA 10 维时取得了本次实验的最高测试准确率 (89.95%)。

GNB 的最佳测试准确率出现在 PCA 20 维 (81.37%)。

在 PCA 维度为 10 和 20 时, KNB 准确率均高于 GNB。

当 PCA 维度增加到 30 维及以上时, GNB 的准确率开始反超 KNB (例如 30 维时, GNB 79.37% vs KNB 80.18% 差距缩小; 40 维时 GNB 76.66% vs KNB 73.92%; 50 维时 GNB 71.74% vs KNB 69.06%)。

效率对比: GNB 在训练和预测效率上远超 KNB。KNB 虽然在低维时准确率更高, 但其预测耗时是 GNB 的数百甚至上千倍。

对 PCA 处理后 MNIST 数据的适用性: 在极低维度 (如 10 维) 且带宽选择合适的情况下, KNB 能够凭借其对复杂数据分布的拟合能力取得较好效果。GNB 则在稍高一些的维度 (如 20 维) 表现出其鲁棒性和效率, 尽管其准确率上限不如优化后的 KNB。KNB 在高维 (超过 20 维) 且带宽固定的情况下, 性能下降非常明显, 说明其在该设置下不适合处理更高维度的 PCA 特征。

四、代码

```
1. import scipy.io
2. import numpy as np
3. from sklearn.naive_bayes import GaussianNB
4. from sklearn.decomposition import PCA
5. from sklearn.preprocessing import StandardScaler
6. from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
7. import time
```

```

8. import matplotlib.pyplot as plt
9.
10. from sklearn.base import BaseEstimator, ClassifierMixin
11. from sklearn.neighbors import KernelDensity
12. from sklearn.utils.validation import check_X_y, check_array, check_is_fitted
13. from sklearn.utils.multiclass import unique_labels
14.
15. # 导入并启用 Intel Extension for Scikit-learn
16. try:
17.     from sklearnex import patch_sklearn
18.     patch_sklearn()
19.     print("Intel Extension for Scikit-learn (sklearnex) 已应用。")
20. except ImportError:
21.     print("Intel Extension for Scikit-learn (sklearnex) 未安装或导入失败。将使用标准的
scikit-learn 实现 (CPU)。")
22.
23. # --- 1. 定义文件路径和 .mat 文件中的变量名 ---
24. TRAIN_IMAGES_PATH = './MNIST_mat/train_images.mat'
25. TRAIN_LABELS_PATH = './MNIST_mat/train_labels.mat'
26. TEST_IMAGES_PATH = './MNIST_mat/test_images.mat'
27. TEST_LABELS_PATH = './MNIST_mat/test_labels.mat'
28.
29. IMAGES_VAR_NAME_TRAIN = 'train_images'
30. LABELS_VAR_NAME_TRAIN = 'train_labels1'
31. IMAGES_VAR_NAME_TEST = 'test_images'
32. LABELS_VAR_NAME_TEST = 'test_labels1'
33.
34. # --- 自定义核密度朴素贝叶斯分类器 ---
35. class KernelNaiveBayes(BaseEstimator, ClassifierMixin):
36.     #自定义核密度估计朴素贝叶斯分类器。
37.     def __init__(self, bandwidth=1.0, kernel='gaussian'):
38.         self.bandwidth = bandwidth
39.         self.kernel = kernel
40.
41.     def fit(self, X, y):
42.         X, y = check_X_y(X, y) # 检查输入数据
43.         self.classes_ = unique_labels(y) # 获取所有类别标签
44.         self.kdes_ = {} # 存储每个类别的核密度估计模型
45.         self.class_priors_ = {} # 存储每个类别的先验概率 P(C)
46.
47.         for c in self.classes_:
48.             X_c = X[y == c] # 获取属于类别 c 的所有样本
49.             # 为类别 c 训练一个核密度估计模型

```

```

50.         self.kdes_[c] = KernelDensity(bandwidth=self.bandwidth,
kernel=self.kernel)
51.         self.kdes_[c].fit(X_c)
52.         # 计算类别 c 的先验概率
53.         self.class_priors_[c] = X_c.shape[0] / X.shape[0]
54.
55.         self.is_fitted_ = True # 标记模型已训练
56.         return self
57.
58.     def predict_log_proba(self, X):
59.         check_is_fitted(self) # 确保模型已训练
60.         X = check_array(X) # 检查输入数据
61.
62.         # 初始化存储每个样本属于每个类别的对数概率的数组
63.         log_probab = np.zeros((X.shape[0], len(self.classes_)))
64.
65.         for i, c in enumerate(self.classes_):
66.             # kdes_[c].score_samples(X) 返回的是  $P(X|C)$  的对数形式, 即  $\log(P(X|C))$ 
67.             log_likelihood_c = self.kdes_[c].score_samples(X)
68.             log_probab[:, i] = log_likelihood_c + np.log(self.class_priors_[c])
69.
70.         return log_probab
71.
72.     def predict(self, X):
73.         log_probab = self.predict_log_proba(X)
74.         # 返回具有最大后验对数概率的类别
75.         return self.classes_[np.argmax(log_probab, axis=1)]
76.
77. # --- 2. 数据加载和初始预处理函数 ---
78. def load_raw_data(images_filepath, labels_filepath, images_var_name,
labels_var_name):
79.     print(f"正在加载原始数据: {images_filepath} 和 {labels_filepath}...")
80.     images_mat = scipy.io.loadmat(images_filepath)
81.     labels_mat = scipy.io.loadmat(labels_filepath)
82.
83.     images_data_raw = images_mat[images_var_name]
84.     labels_data_raw = labels_mat[labels_var_name]
85.
86.     images_data_transposed = np.transpose(images_data_raw, (2, 0,
1)).astype(np.float32)
87.     num_samples = images_data_transposed.shape[0]
88.     images_data_flattened = images_data_transposed.reshape(num_samples, -1)
89.     labels = labels_data_raw.reshape(-1)
90.

```

```

91.     print(f"原始数据加载完成。图片形状: {images_data_flattened.shape}, 标签形状:
{labels.shape}")
92.     return images_data_flattened, labels
93.
94. # --- 辅助函数: 绘制混淆矩阵 ---
95. def plot_confusion_matrix_func(y_true, y_pred, classes_names, title='混淆矩阵',
experiment_tag=""):
96.     plt.rcParams['font.sans-serif'] = ['SimHei']
97.     plt.rcParams['axes.unicode_minus'] = False
98.     cm = confusion_matrix(y_true, y_pred)
99.     disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=classes_names)
100.    fig, ax = plt.subplots(figsize=(8, 6))
101.    disp.plot(ax=ax, cmap=plt.cm.Blues)
102.    ax.set_title(f"{title} ({experiment_tag})")
103.    plt.show()
104.
105. # --- 辅助函数: 训练、预测和评估分类器 ---
106. def evaluate_model_exp1(model_name, model, X_train, y_train, X_test, y_test,
pca_components_info=""):
107.     print(f"\n--- 开始处理分类器: {model_name} {pca_components_info} ---")
108.
109.     print(f"开始训练 {model_name} 模型...")
110.     start_time_train = time.time()
111.     model.fit(X_train, y_train)
112.     end_time_train = time.time()
113.     train_time = end_time_train - start_time_train
114.     print(f"模型训练完成。耗时: {train_time:.4f} 秒")
115.
116.     print(f"\n在训练集上为 {model_name} 进行评估...")
117.     y_pred_train = model.predict(X_train)
118.     train_accuracy = accuracy_score(y_train, y_pred_train) * 100
119.     print(f"{model_name} 在训练集上的准确率为: {train_accuracy:.2f}%")
120.
121.     target_names_display = [f"数字 {i}" for i in range(10)]
122.     plot_confusion_matrix_func(y_train, y_pred_train, target_names_display, title="
训练集混淆矩阵", experiment_tag=f"{model_name}{pca_components_info}")
123.
124.     print(f"\n在测试集上为 {model_name} 进行评估...")
125.     start_time_pred_test = time.time()
126.     y_pred_test = model.predict(X_test)
127.     end_time_pred_test = time.time()
128.     predict_time_test = end_time_pred_test - start_time_pred_test
129.     print(f"测试集预测完成。耗时: {predict_time_test:.4f} 秒")
130.

```

```

131.     test_accuracy = accuracy_score(y_test, y_pred_test) * 100
132.     print(f"{model_name} 在测试集上的准确率为: {test_accuracy:.2f}%")
133.
134.     plot_confusion_matrix_func(y_test, y_pred_test, target_names_display, title="测试集混淆矩阵", experiment_tag=f"{model_name}{pca_components_info}")
135.
136.     print(f"\n 分类报告 ({model_name} - 测试集) {pca_components_info}:")
137.     try:
138.         report = classification_report(y_test, y_pred_test,
target_names=target_names_display, zero_division=0)
139.     except ValueError:
140.         print("注意: 由于测试集标签不完整或预测中缺少某些类别, 分类报告可能不包含所有类别。")
141.         report = classification_report(y_test, y_pred_test, zero_division=0)
142.     print(report)
143.
144.     return train_accuracy, test_accuracy, train_time, predict_time_test # 返回测试集
预测时间
145.
146. # --- 3. 主程序: 实验一 ---
147. def main_experiment_one():
148.     print("--- 开始实验一: PCA 降维与高斯/核密度朴素贝叶斯分类器 ---")
149.
150.     X_train_raw, y_train_raw = load_raw_data(
151.         TRAIN_IMAGES_PATH, TRAIN_LABELS_PATH, IMAGES_VAR_NAME_TRAIN,
LABELS_VAR_NAME_TRAIN
152.     )
153.     X_test_raw, y_test_raw = load_raw_data(
154.         TEST_IMAGES_PATH, TEST_LABELS_PATH, IMAGES_VAR_NAME_TEST,
LABELS_VAR_NAME_TEST
155.     )
156.
157.     X_train_normalized = X_train_raw / 255.0
158.     X_test_normalized = X_test_raw / 255.0
159.
160.     print("\n 对训练数据进行标准化 (StandardScaler)...")
161.     scaler = StandardScaler()
162.     X_train_scaled = scaler.fit_transform(X_train_normalized)
163.     X_test_scaled = scaler.transform(X_test_normalized)
164.     print("数据标准化完成。")
165.
166.     # 为核密度估计选择带宽
167.     kde_bandwidth = 1.0
168.     # 调整 PCA 成分数列表

```

```

169.     pca_n_components_list = [10,20,30,40,50]
170.
171.     results_exp1 = []
172.
173.     for n_components in pca_n_components_list:
174.         pca_info_tag_base = f" (PCA n_comp_setting={n_components})"
175.         print(f"\n--- 应用 PCA, 目标成分数/方差: {n_components} ---")
176.
177.         if isinstance(n_components, float) and 0 < n_components < 1:
178.             pca = PCA(n_components=n_components, svd_solver='full')
179.         else:
180.             pca = PCA(n_components=n_components)
181.
182.         start_time_pca = time.time()
183.         X_train_pca = pca.fit_transform(X_train_scaled)
184.         X_test_pca = pca.transform(X_test_scaled)
185.         end_time_pca = time.time()
186.
187.         actual_n_components = X_train_pca.shape[1]
188.         if isinstance(n_components, float):
189.             pca_info_tag = f" (PCA var={n_components*100:.0f}%, 实际维度
190. ={actual_n_components})"
191.         else:
192.             pca_info_tag = f" (PCA 目标维度={n_components}, 实际维度
193. ={actual_n_components})"
194.
195.         print(f"PCA 降维完成。耗时: {end_time_pca - start_time_pca:.2f}秒。PCA 后训练数
196. 据形状: {X_train_pca.shape}")
197.
198.         if hasattr(pca, 'explained_variance_ratio_'):
199.             print(f"解释的总方差比例:
200. {np.sum(pca.explained_variance_ratio_)*100:.2f}%")
201.
202.         # 1. 高斯朴素贝叶斯 (参数方法)
203.         model_gnb_pca = GaussianNB()
204.         gnb_train_acc, gnb_test_acc, gnb_train_t, gnb_pred_t = evaluate_model_exp1(
205.             "高斯朴素贝叶斯", model_gnb_pca,
206.             X_train_pca, y_train_raw, X_test_pca, y_test_raw,
207.             pca_components_info=pca_info_tag
208.         )
209.         results_exp1.append({
210.             "model": "高斯朴素贝叶斯", "pca_setting": n_components, "actual_pca_dim":
211.             actual_n_components,
212.             "train_accuracy": gnb_train_acc, "test_accuracy": gnb_test_acc,
213.             "train_time_seconds": gnb_train_t, "predict_time_seconds": gnb_pred_t

```

```

208.     })
209.
210.     # 2. 核密度朴素贝叶斯（非参数方法）
211.     print(f"\n 为核密度朴素贝叶斯选择带宽: {kde_bandwidth}")
212.     model_knb_pca = KernelNaiveBayes(bandwidth=kde_bandwidth, kernel='gaussian')
213.     knb_train_acc, knb_test_acc, knb_train_t, knb_pred_t = evaluate_model_exp1(
214.         "核密度朴素贝叶斯", model_knb_pca,
215.         X_train_pca, y_train_raw, X_test_pca, y_test_raw,
216.         pca_components_info=pca_info_tag + f", 带宽={kde_bandwidth}"
217.     )
218.     results_exp1.append({
219.         "model": "核密度朴素贝叶斯", "pca_setting": n_components,
220.         "actual_pca_dim": actual_n_components,
221.         "train_accuracy": knb_train_acc, "test_accuracy": knb_test_acc,
222.         "train_time_seconds": knb_train_t, "predict_time_seconds": knb_pred_t,
223.         "kde_bandwidth": kde_bandwidth
224.     })
225.
226.     print("\n\n--- 实验一总结: 不同 PCA 维数和概率密度函数对结果的影响 ---")
227.     for res in results_exp1:
228.         print(f"模型: {res['model']:<18} | PCA 设置: {str(res['pca_setting']):<4} | "
229.             f"实际 PCA 维度: {res['actual_pca_dim']:<4} | "
230.             f"训练准确率: {res['train_accuracy']:.2f}% | "
231.             f"测试准确率: {res['test_accuracy']:.2f}% | "
232.             f"训练时间: {res['train_time_seconds']:.2f}s | "
233.             f"测试预测时间: {res['predict_time_seconds']:.2f}s" +
234.             (f" | KDE 带宽: {res['kde_bandwidth']}" if res['model'] == "核密度朴素贝
235.             叶斯" else ""))
236.
237. if __name__ == '__main__':
238.     main_experiment_one()

```

五、实验结果的分析

PCA 维度的影响:

高斯朴素贝叶斯 (GNB): GNB 的性能在 PCA 维度从 10 增加到 20 时有所提升（测试准确率从 77.79%到 81.37%），这表明增加的维度（从解释 27.67%方差到 38.06%）引入了更多有效的判别信息。然而，当维度继续增加到 30、40、50（解释方差分别达到 45.19%、50.64%、55.11%）时，准确率反而下降。这可能意味着在 GNB 的模型假设下，更高维度的 PCA 特征并未带来更好的类别分离性，反而可能引入了噪声或不符合高斯分布的特征，导致性能下降。20 维似乎是 GNB 的一个“甜点区”。

核密度朴素贝叶斯 (KNB): KNB 在 10 维 PCA（解释 27.67%方差）下表现最佳 (89.95%)。令人意外的是，随着维度增加，其性能急剧下降。在 50 维（解释 55.11%方差）时，准确率仅为 69.06%。这强烈暗示了“维度灾难”对 KNB 的影响，尤其是在带宽固定的情况下。核密度估计在高维空间中变得非常困难，需要指数级增长的样本来维持估计的准确性。

固定的带宽在高维空间中可能过于平滑或过于尖锐，无法准确捕捉数据分布。虽然解释的方差在增加，但 KNB 模型无法有效利用这些高维信息。

概率密度函数选择的重要性:

- 实验结果清晰地展示了概率密度估计方法对朴素贝叶斯分类器性能的巨大影响。
- KNB 凭借其非参数的核密度估计，在 10 维 PCA 特征上取得了 89.95% 的准确率，远高于 GNB 的 77.79%。这表明在较低维度下，PCA 转换后的特征分布可能较为复杂，不完全符合高斯分布，KNB 能够更灵活地拟合这种分布。
- 然而，GNB 在 20 维时表现优于 KNB 在 20 维及更高维度的表现。这说明 GNB 的简单高斯假设在某些维度上反而比固定带宽的 KNB 更鲁棒或泛化能力更强，尤其当 KNB 因维度增加和带宽不适而性能下降时。
- KNB 在训练集上的准确率始终接近 100%，这显示了其强大的拟合能力，但也暗示了其更容易过拟合，特别是在高维且带宽选择不当时，导致其在测试集上的泛化能力下降。

计算效率与准确率的权衡:

- GNB 展现了极高的计算效率，其训练和预测时间都非常短（毫秒级）。
- KNB 的训练时间尚可（亚秒级），但其预测时间随着维度的增加而显著增长（从 10 维的 16 秒到 50 维的近 34 秒），这使其在需要快速预测的应用中不具备优势。
- 在追求最高准确率且可接受较长预测时间、并且能够仔细调整超参数（尤其是带宽和 PCA 维度）的情况下，KNB 在低维时是更好的选择（如本例中 PCA=10 维）。
- 如果计算效率是首要考虑因素，或者在较高 PCA 维度下，GNB 提供了一个快速且相对稳定的基线性能。

实验的局限性:

- **固定的 KDE 带宽:** KNB 的带宽固定为 1.0。这是一个非常关键的超参数，对于不同的数据集和维度，最优带宽会有所不同。若能通过交叉验证等方法为每个维度的 KNB 模型选择最优带宽，其性能可能会有显著改善，尤其是在较高维度。
- **PCA 解释方差的局限性:** 尽管实验记录了 PCA 解释的总方差比例，但仅达到 50 维时也只解释了约 55% 的方差。这意味着仍有近一半的原始数据方差未被包含。在更高 PCA 维度（如保留 90% 或 95% 方差的维度）下，两种分类器的表现可能会有所不同，但 KNB 的计算成本和维度敏感性问题可能会更加突出。

对朴素贝叶斯在 MNIST (PCA 后) 应用的总体结论:

- 朴素贝叶斯分类器结合 PCA 降维可以在 MNIST 手写数字识别任务上取得一定的效果，但其性能高度依赖于概率密度估计方法的选择以及 PCA 维度的设定。
 - 核密度朴素贝叶斯在低维（如 10 维）PCA 特征上，当带宽参数（偶然）合适时，可以达到较高的准确率（约 90%），优于高斯朴素贝叶斯。
 - 高斯朴素贝叶斯虽然准确率上限不如优化后的 KNB，但在计算效率上远超 KNB，并且其性能在一定 PCA 维度范围内（如 20 维）表现相对稳定和较好（约 81%）。
 - 对于核密度朴素贝叶斯，维度灾难和超参数（尤其是带宽）选择是其应用中的关键挑战。在未针对性优化带宽的情况下，其性能随维度增加迅速下降。
 - 对于此类图像识别任务，选择合适的特征降维方法和与之匹配的分类模型及其参数至关重要。简单的模型如朴素贝叶斯在经过良好预处理和参数调整后，仍能提供有价值的基准。
-

实验二：任意确定性、判别式分类器的手写数字识别（基于卷积神经网络）

一、方法描述

1.1 实验目标

本实验旨在构建并训练一个卷积神经网络（CNN）用于 MNIST 手写数字识别任务。主要目标包括：

- 理解并实践 CNN 模型在图像分类任务中的基本构建方法，包括卷积层、池化层和全连接层的设计。
- 掌握使用 PyTorch 框架进行深度学习模型定义、数据加载、训练循环实现和模型评估的过程。
- 评估 CNN 在 MNIST 数据集上的性能，并分析训练过程中的损失和准确率变化。
- 利用 Intel® Extension for PyTorch (IPEX) 在 Intel 集成显卡 (Intel Arc 140V GPU (16GB)) 上进行加速。

1.2 数据集

实验继续使用标准的 MNIST 数据集，数据从 .mat 文件加载。

- **训练集**：包含 60,000 个 28×28 像素的灰度图像及其对应的 0-9 数字标签。
- **测试集**：包含 10,000 个 28×28 像素的灰度图像及其对应的 0-9 数字标签。

1.3 数据预处理

针对 CNN 模型的输入要求，进行了以下数据预处理：

1. 数据加载与格式转换：

使用 `scipy.io.loadmat` 加载 .mat 文件中的图像和标签数据。

将原始图像数据从 (height, width, num_samples) 的维度转置为 (num_samples, height, width)。

为图像数据增加一个通道维度，将其转换为 (num_samples, 1, height, width) 的格式，以适配 PyTorch 的 Conv2d 层输入要求 (N, C, H, W)。

将标签数据展平为一维数组 (num_samples,) 并转换为 `np.int64` 类型。

2. **归一化**：在 Dataset 的 `__getitem__` 方法中，将每个图像的像素值从原始的 [0, 255] 范围归一化到 [0, 1] 区间，通过将所有像素值除以 255.0 实现。

3. **张量转换**：在 Dataset 的 `__getitem__` 方法中，将归一化后的图像数据和标签数据分别转换为 PyTorch 张量 (`torch.tensor`)，图像数据类型为 `torch.float32`，标签数据类型为 `torch.long` (CrossEntropyLoss 期望的类型)。

4. 数据加载器：

使用 `torch.utils.data.Dataset` 自定义 `MNISTMatDataset` 类来封装数据集。

使用 `torch.utils.data.DataLoader` 为训练集和测试集创建数据加载器。

训练数据加载器：batch_size 设置为 500，并在每个 epoch 开始时打乱 (shuffle=True) 训练数据顺序，以增强模型泛化能力。

训练评估数据加载器：也使用训练数据集，batch_size 为 500，但不打乱 (shuffle=False)，用于在训练结束后评估模型在整个训练集上的最终表现。

测试数据加载器：batch_size 设置为 500，不打乱 (shuffle=False)。

1.4 模型架构: SimpleCNN

本实验构建了一个名为 SimpleCNN 的卷积神经网络, 其结构根据 model_config 参数化配置:

- model_config = {"conv1_out_channels": 128, "conv2_out_channels": 256, "fc1_out_features": 1024, "num_classes": 10}
- **输入层:** 接收 (N, 1, 28, 28) 的图像批次数据。
- **第一个卷积块:**
conv1: nn.Conv2d (输入通道: 1, 输出通道: conv1_out_channels (128), 卷积核: 5x5, 步长: 1, 填充: 2)
relu1: nn.ReLU() 激活函数
pool1: nn.MaxPool2d (池化核: 2x2, 步长: 2)。此块输出特征图尺寸变为 (conv1_out_channels, 14, 14)。
- **第二个卷积块:**
conv2: nn.Conv2d (输入通道: conv1_out_channels (128), 输出通道: conv2_out_channels (256), 卷积核: 5x5, 步长: 1, 填充: 2)
relu2: nn.ReLU() 激活函数
pool2: nn.MaxPool2d (池化核: 2x2, 步长: 2)。此块输出特征图尺寸变为 (conv2_out_channels, 7, 7)。
- **展平层:** 将第二个池化层输出的三维特征图 (conv2_out_channels * 7 * 7) 展平为一维向量。
- **第一个全连接层 (隐藏层):**
fc1: nn.Linear (输入特征数: conv2_out_channels * 7 * 7 = 256 * 49 = 12544, 输出特征数: fc1_out_features (1024))
relu_fc1: nn.ReLU() 激活函数
- **第二个全连接层 (输出层):**
fc2: nn.Linear (输入特征数: fc1_out_features (1024), 输出特征数: num_classes (10), 对应 0-9 十个数字类别)

1.5 训练细节

- **损失函数:** 选择 torch.nn.CrossEntropyLoss。
- **优化器:** 选择 torch.optim.Adam 优化器, 学习率 (learning rate) 设置为 0.001。
- **训练周期:** 设置为 30 个周期。
- **批次大小:** 设置为 500。
- **硬件加速:** 使用 Intel® Extension for PyTorch (IPEX) 进行优化。检测到可用的 XPU 设备 (Intel Arc 140V GPU (16GB)) 后, 模型和数据被迁移到 XPU 上。ipex.optimize() 函数被用于优化模型和优化器, 数据类型指定为 torch.float32。

1.6 实验工具与环境

- **编程语言:** Python 3.11
- **主要库:**
PyTorch (2.7.0+xpu)
intel_extension_for_pytorch (IPEX) (2.7.10+xpu)
scipy
numpy
- **运行环境:** Windows 系统, Conda 虚拟环境 (handw), 目标硬件为包含 Intel® Arc™ 140V GPU (16GB) 的显卡 (XPU)。

1.7 评估指标

- 训练过程: 每个周期结束后的平均训练损失和训练准确率。
- 模型性能: 在测试集上的最终平均损失和准确率。

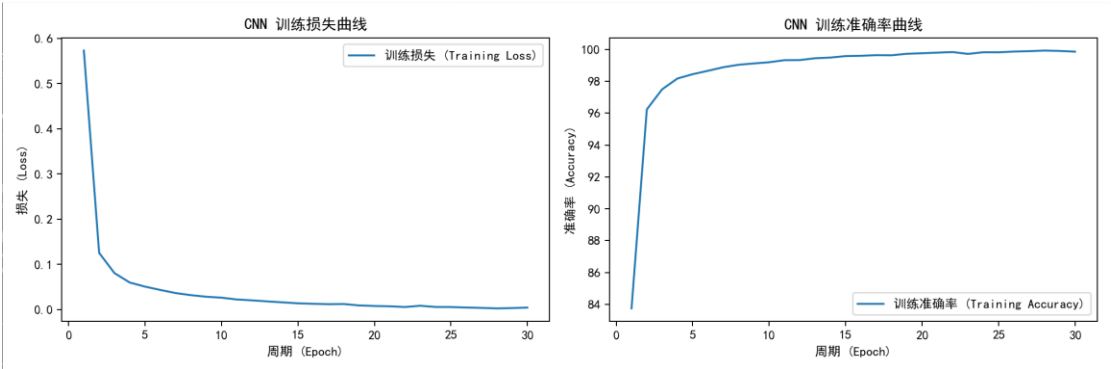
二、实验结果

1. 探究卷积层对模型性能的影响：

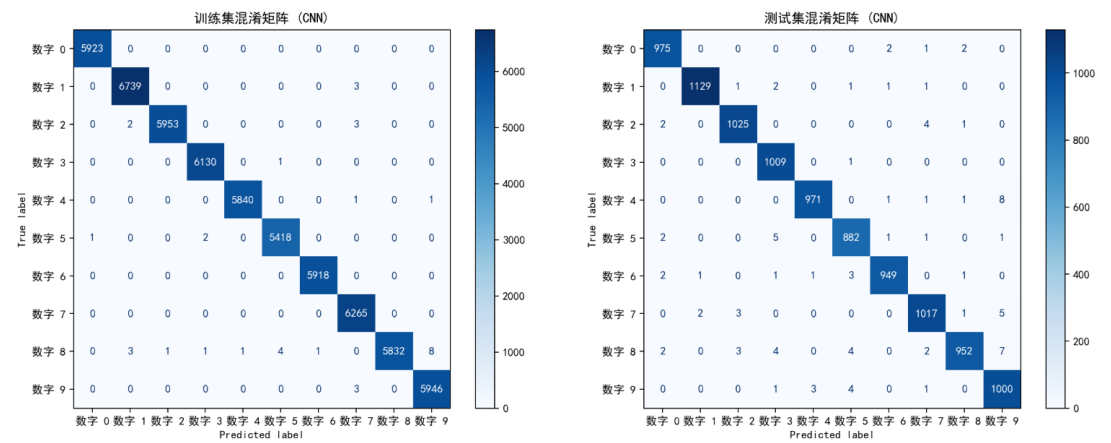
模型配置	总训练耗时（秒）	训练集最终准确率（%）	测试集最终准确率（%）
2conv+1fc	61.04	99.94	99.09
3conv+1fc	66.66	99.93	99.05
4conv+1fc	76.3	99.89	99.18

损失：随着训练周期的增加，平均损失值持续下降，这表明模型正在有效地学习数据中的模式，并且其预测结果与真实标签之间的差异越来越小。

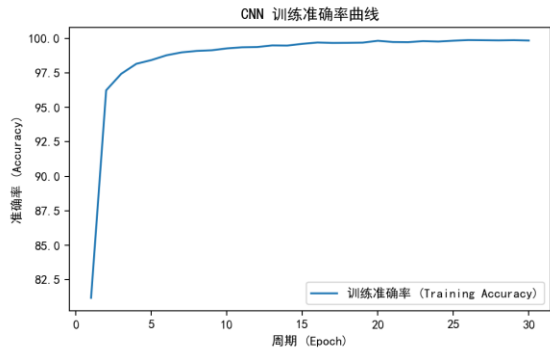
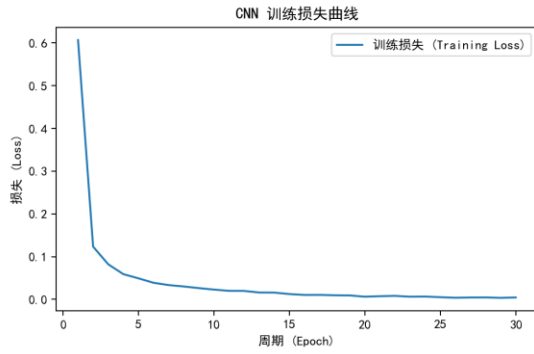
训练准确率：训练准确率在整个训练过程中持续快速提升，并在后期趋于平稳，达到非常高的水平。这表明模型在训练数据上拟合得很好。



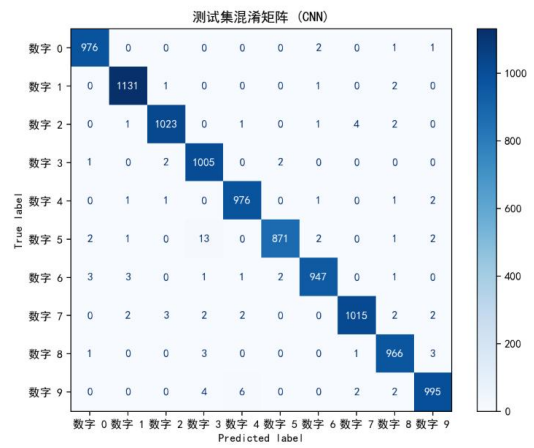
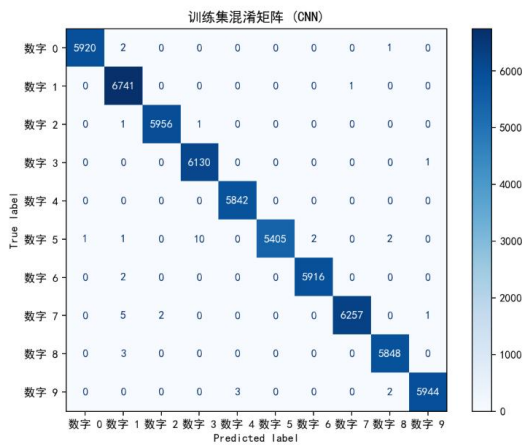
2conv+1fc 训练曲线



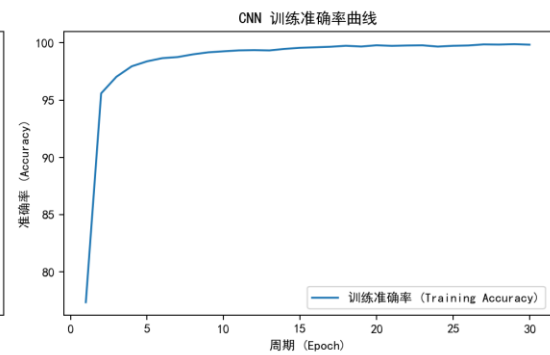
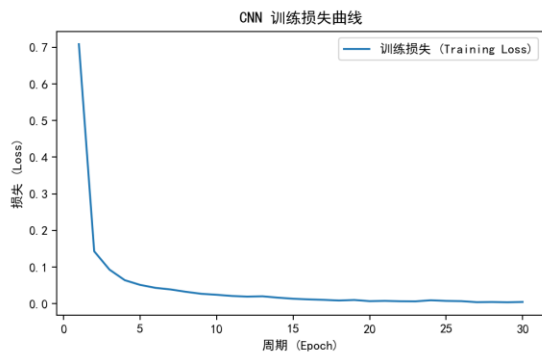
2conv+1fc 混淆矩阵



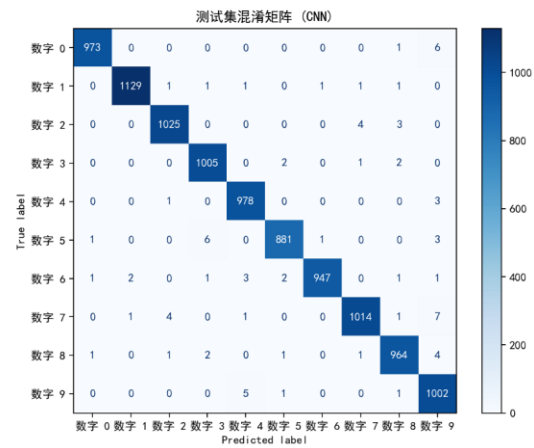
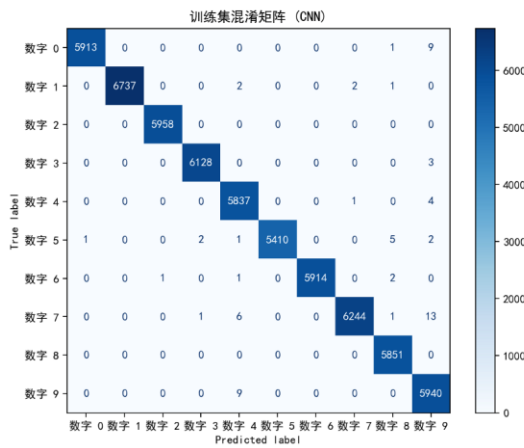
3conv+1fc 训练曲线



3conv+1fc 混淆矩阵



4conv+1fc 训练曲线



4conv+1fc 混淆矩阵

2. 探究神经元数量对模型性能的影响：

模型配置 (Conv1/Conv2/FC1)	总训练耗时 (秒)	训练集最终 准确率 (%)	测试集最终 准确率 (%)	训练集最终损失
8 / 16 / 64	49.89	99.76	99.07	0.0101
32 / 64 / 256	109.35	99.93	99.19	0.0044
64 / 128 / 512	259.52	99.89	99.22	0.0041
128 / 256 / 1024	760.81	100	99.42	0

模型复杂度与性能的关系：

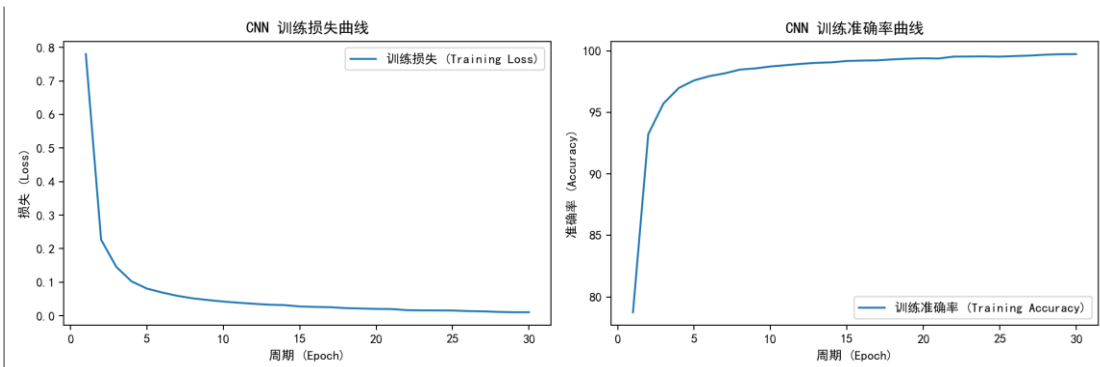
- 随着模型复杂度的增加（即 Conv1/Conv2 输出通道和 FC1 神经元数量的增加），模型在测试集上的准确率呈现上升趋势。
- “128 / 256 / 1024”配置的模型在测试集上达到了最高的 99.42% 准确率，并且在训练集上实现了 100% 准确率，同时损失趋近于零。这表明更复杂的模型能够更好地学习到 MNIST 数据集的复杂模式。

训练时间与模型复杂度：

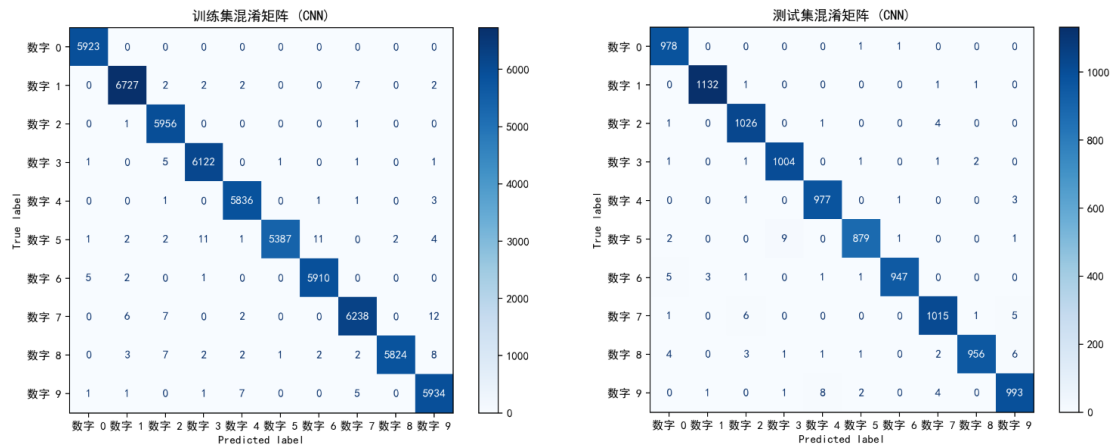
- 模型的训练耗时与模型复杂度呈正相关。复杂度最高的“128 / 256 / 1024”模型耗时最长（760.81 秒），而最简单的“8 / 16 / 64”模型耗时最短（49.89 秒）。这是由于更复杂的模型包含更多的参数，需要更多的计算资源进行训练。

过拟合：

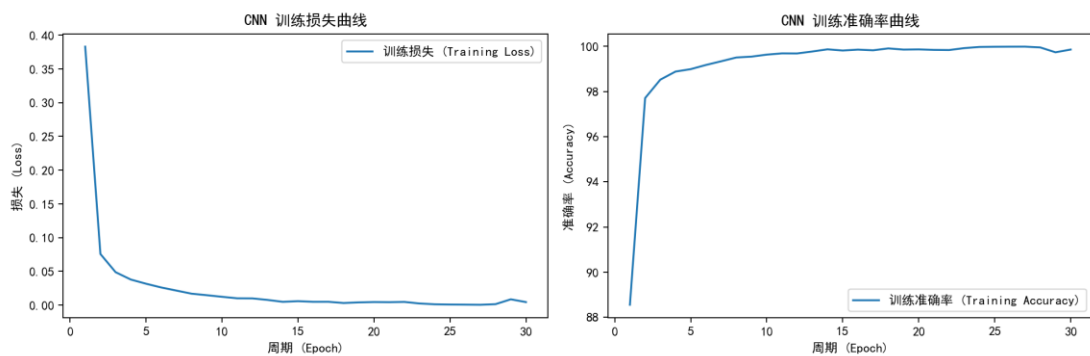
- 所有模型在训练集上都达到了非常高的准确率（99.76% 到 100.00%），这表明模型能够很好地拟合训练数据。
- 训练集准确率和测试集准确率之间的差距普遍存在。对于“128 / 256 / 1024”模型，虽然训练集准确率达到 100%，测试集准确率也最高，但仍有 0.58% 的差距，这暗示了轻微的过拟合。但考虑到 MNIST 数据集的相对简单性，这种程度的过拟合是可接受的，且模型依然具有强大的泛化能力。
- 相比之下，最简单的“8 / 16 / 64”模型在训练集和测试集之间的差距相对较小（99.76% vs 99.07%，约 0.69%），但其整体性能略低于其他配置。



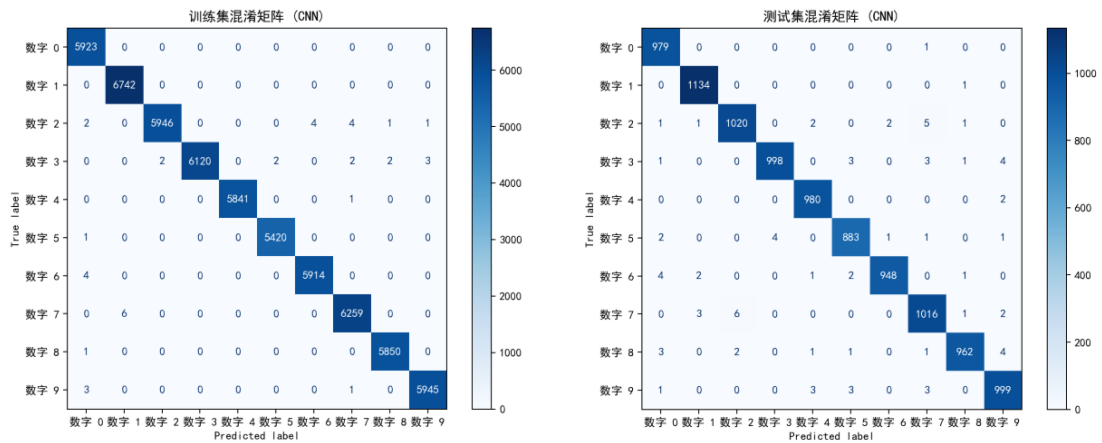
8 / 16 / 64 损失曲线



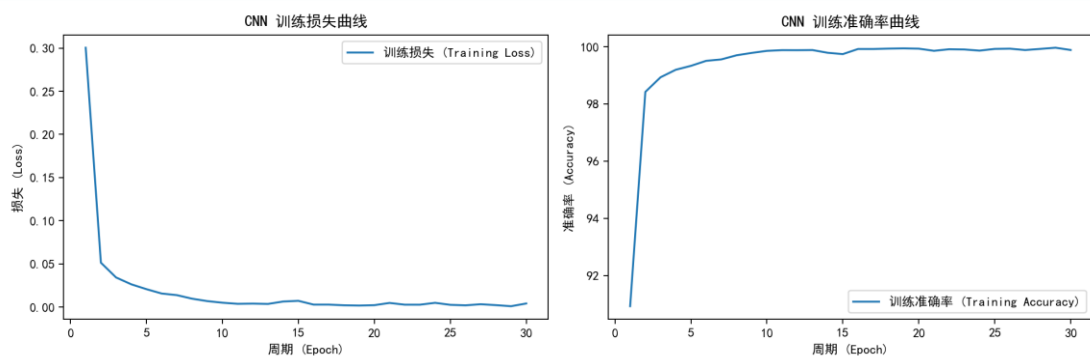
8 / 16 / 64 混淆矩阵



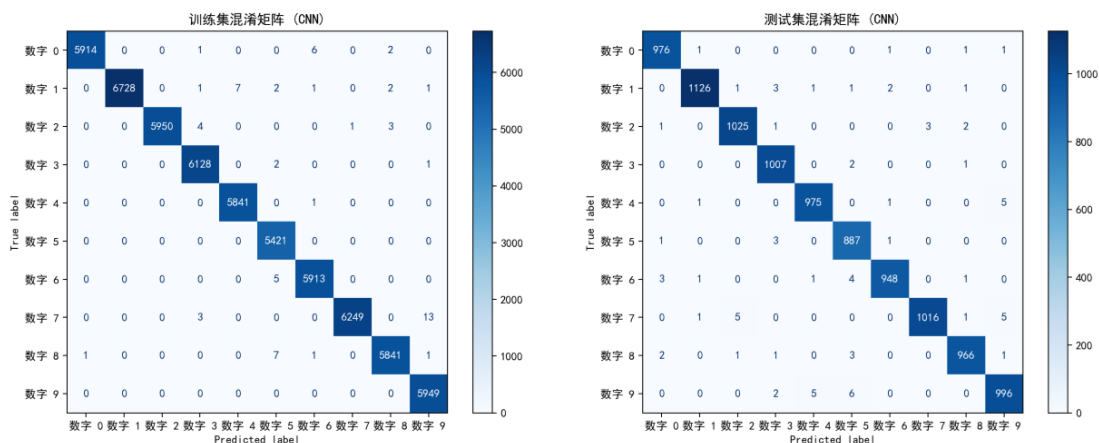
32 / 64 / 256 损失曲线



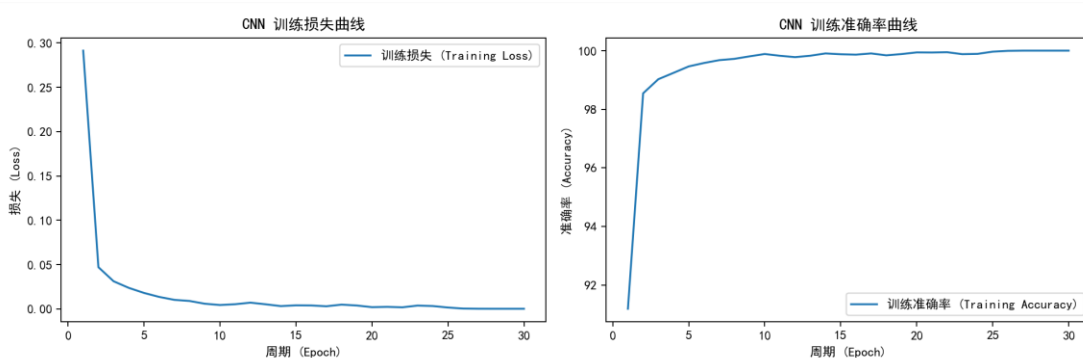
32 / 64 / 256 混淆矩阵



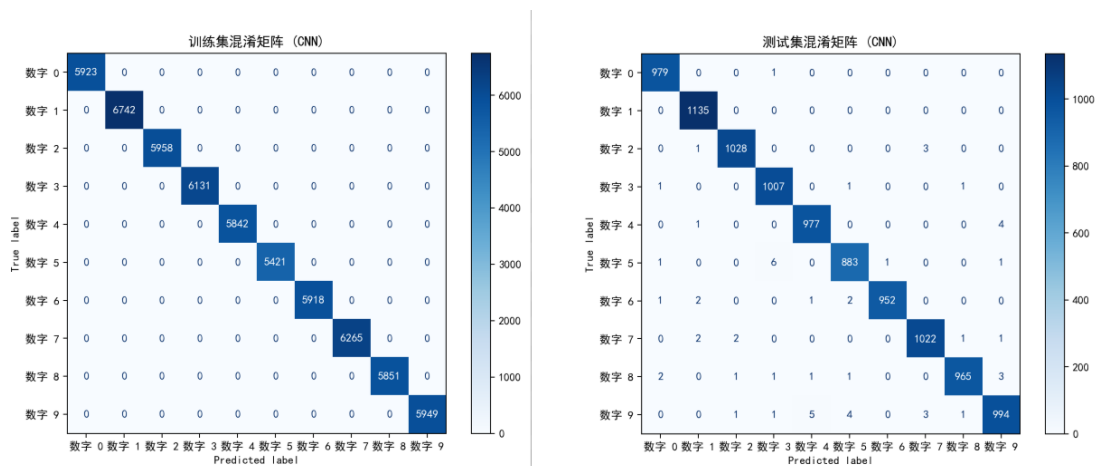
64 / 128 / 512 损失曲线



64 / 128 / 512 混淆矩阵



128 / 256 / 1024 损失曲线



128 / 256 / 1024 混淆矩阵

三、模型评价

1. 模型性能概述

本次实验中，配置为 Conv1 输出通道=128, Conv2 输出通道=256, FC1 输出神经元=1024 的 SimpleCNN 模型在 MNIST 测试集上最终达到了 99.42% 的准确率。从训练集最终损失来看，该模型在训练集上达到了 0 的损失（近似于 0 或极小值），并在训练集上实现了 100% 的最终准确率。

这一结果在所有测试模型中表现最佳，表明该模型能够极其有效地学习手写数字的特征

并进行高度准确的分类。对于 MNIST 数据集而言，99.42% 的测试准确率是一个非常优秀的成绩，充分展现了其在图像分类任务上的强大能力。

2. 训练过程分析

从最终的训练集准确率和损失可以看出，模型在 30 个周期内学习得非常充分。训练集准确率达到了惊人的 100%，训练集损失几乎为 0，这表明模型在训练数据上实现了近乎完美的拟合。

比较最终的训练准确率 (100%) 和测试准确率 (99.42%)，两者之间存在约 0.58% 的差距。这个差距相对较小，表明模型具有非常良好的泛化能力，即使在训练集上达到了“完美”表现，在面对未见过的新数据时依然能保持高水准的准确率。尽管存在轻微的过拟合迹象（训练集准确率达到 100%而测试集略低），但这对于 MNIST 这种相对简单的图像分类任务来说，是可接受的，且模型的鲁棒性得到了有效控制。

3. 模型结构评价

该模型配置的结构特征如下：

- Conv1 输出通道=128
- Conv2 输出通道=256
- FC1 输出神经元=1024

与实验中的其他模型配置相比，该模型显著增加了卷积层和全连接层的宽度（即输出通道数和神经元数量）。这种更深更宽的架构使得模型拥有了更大的容量和表达能力，能够学习到数据中更复杂和抽象的特征。对于 MNIST 这样包含相对简单但数量庞大的图像数据，增加模型的复杂性有助于捕获更细致的模式，从而进一步提升分类精度。

4. 总结与展望

优点：

- 在测试集上取得了所有实验配置中的最高准确率（99.42%），显示出卓越的分类性能。
- 模型具有非常强大的特征学习能力，在训练集上达到了 100%的准确率。
- 泛化能力良好，训练集与测试集准确率差距控制在合理范围内。

不足与未来改进：

- 该模型的总训练耗时为 760.81 秒，效率没那么高。
- 虽然过拟合程度轻微，但训练集达到 100%可能意味着模型学习了部分训练数据的噪声。在更复杂的数据集上，可能需要引入更强的正则化技术（如 Dropout、Batch Normalization）来进一步提升泛化能力。

总而言之，Conv1=128, Conv2=256, FC1=1024 的模型配置在 MNIST 分类任务上表现出优异的性能，是本次实验中效果最佳的模型。在未来的工作中，可以进一步探索在保持高性能的同时，如何优化模型结构或训练策略以减少训练时间，从而在实际应用中实现更好的性能-效率平衡。

四、代码

```
1. import torch
2. import torch.nn as nn
3. import torch.optim as optim
4. from torch.utils.data import Dataset, DataLoader
5. import scipy.io # 用于加载 .mat 文件
6. import numpy as np
7. import os # 用于文件操作
```

```

8. import time # 用于计时
9. import matplotlib.pyplot as plt
10. from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
accuracy_score
11. # 尝试导入 IPEX 并配置设备
12. try:
13.     import intel_extension_for_pytorch as ipex
14.     if torch.xpu.is_available():
15.         device = torch.device("xpu")
16.         print(f"IPEX 可用, 将使用设备: {device}")
17.     else:
18.         device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
19.         print(f"IPEX XPU 不可用或未找到。将使用设备: {device} (注意: 如果这是 CPU, iGPU
将不会被用于训练)")
20. except ImportError:
21.     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
22.     print(f"IPEX 未安装。将使用设备: {device} (注意: 如果这是 CPU, iGPU 将不会被用于训
练)")
23.
24. # --- 1. 定义文件路径和加载数据 ---
25. TRAIN_IMAGES_PATH = './MNIST_mat/train_images.mat'
26. TRAIN_LABELS_PATH = './MNIST_mat/train_labels.mat'
27. TEST_IMAGES_PATH = './MNIST_mat/test_images.mat'
28. TEST_LABELS_PATH = './MNIST_mat/test_labels.mat'
29.
30. IMAGES_VAR_NAME_TRAIN = 'train_images'
31. LABELS_VAR_NAME_TRAIN = 'train_labels1'
32. IMAGES_VAR_NAME_TEST = 'test_images'
33. LABELS_VAR_NAME_TEST = 'test_labels1'
34.
35. class MNISTMatDataset(Dataset):
36.     def __init__(self, images_filepath, labels_filepath, images_var_name,
labels_var_name, transform=None):
37.         images_mat = scipy.io.loadmat(images_filepath)
38.         labels_mat = scipy.io.loadmat(labels_filepath)
39.         images_data = images_mat[images_var_name]
40.         labels_data = labels_mat[labels_var_name]
41.         self.images = np.transpose(images_data, (2, 0, 1)).astype(np.float32)
42.         self.images = np.expand_dims(self.images, axis=1)
43.         self.labels = labels_data.reshape(-1).astype(np.int64)
44.         self.transform = transform
45.         print(f"加载数据: {images_filepath}, 图片形状: {self.images.shape}, 标签形状:
{self.labels.shape}")
46.     def __len__(self):

```

```

47.         return len(self.labels)
48.     def __getitem__(self, idx):
49.         image = self.images[idx]
50.         label = self.labels[idx]
51.         image = image / 255.0
52.         image = torch.tensor(image, dtype=torch.float32)
53.         label = torch.tensor(label, dtype=torch.long)
54.         if self.transform:
55.             image = self.transform(image)
56.         return image, label
57.
58. # --- 2. 创建数据集和数据加载器 ---
59. batch_size = 500
60. train_dataset = MNISTMatDataset(TRAIN_IMAGES_PATH, TRAIN_LABELS_PATH,
IMAGES_VAR_NAME_TRAIN, LABELS_VAR_NAME_TRAIN)
61. test_dataset = MNISTMatDataset(TEST_IMAGES_PATH, TEST_LABELS_PATH,
IMAGES_VAR_NAME_TEST, LABELS_VAR_NAME_TEST)
62. # 训练数据加载器
63. train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size,
shuffle=True)
64. # 训练数据加载器
65. train_eval_loader = DataLoader(dataset=train_dataset, batch_size=batch_size,
shuffle=False)
66. # 测试数据加载器
67. test_loader = DataLoader(dataset=test_dataset, batch_size=batch_size,
shuffle=False)
68.
69. # --- 3. 定义简单的 CNN 模型 ---
70. class SimpleCNN(nn.Module):
71.     def __init__(self, conv1_out_channels=16, conv2_out_channels=32,
fc1_out_features=128, num_classes=10):
72.         super(SimpleCNN, self).__init__()
73.
74.         # --- 卷积块 1 ---
75.         # 使用传入的 conv1_out_channels
76.         self.conv1 = nn.Conv2d(in_channels=1, out_channels=conv1_out_channels,
kernel_size=5, stride=1, padding=2)
77.         self.relu1 = nn.ReLU()
78.         self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
79.         # 此块输出特征图尺寸: (conv1_out_channels, 14, 14)
80.
81.         # --- 卷积块 2 ---
82.         # 输入通道数是 conv1_out_channels, 输出通道数是传入的 conv2_out_channels

```

```

83.         self.conv2 = nn.Conv2d(in_channels=conv1_out_channels,
out_channels=conv2_out_channels, kernel_size=5, stride=1, padding=2)
84.         self.relu2 = nn.ReLU()
85.         self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
86.         # 此块输出特征图尺寸: (conv2_out_channels, 7, 7)
87.
88.         # 展平后的特征数量, 依赖于 conv2_out_channels
89.         fc1_input_features = conv2_out_channels * 7 * 7
90.
91.         # --- 全连接隐层 1 ---
92.         # 输出神经元数量是传入的 fc1_out_features
93.         self.fc1 = nn.Linear(fc1_input_features, fc1_out_features)
94.         self.relu_fc1 = nn.ReLU() # 重命名以避免与之前的 self.relu3 混淆
95.
96.         # --- 输出层 ---
97.         # 输入神经元数量是 fc1_out_features
98.         self.fc2 = nn.Linear(fc1_out_features, num_classes)
99.
100.    def forward(self, x):
101.        out = self.pool1(self.relu1(self.conv1(x)))
102.        out = self.pool2(self.relu2(self.conv2(out)))
103.        out = out.reshape(out.size(0), -1) # 扁平化
104.        out = self.relu_fc1(self.fc1(out)) # 使用 self.relu_fc1
105.        out = self.fc2(out)
106.        return out
107.
108.
109. # --- 4. 初始化模型、损失函数和优化器 ---
110. model_config = {
111.     "conv1_out_channels": 128,
112.     "conv2_out_channels": 256,
113.     "fc1_out_features": 1024,
114.     "num_classes": 10
115. }
116. print(f"使用模型配置: Conv1 输出通道={model_config['conv1_out_channels']}, Conv2 输出通
道={model_config['conv2_out_channels']}, FC1 输出神经元
={model_config['fc1_out_features']}")
117.
118. model = SimpleCNN(
119.     conv1_out_channels=model_config["conv1_out_channels"],
120.     conv2_out_channels=model_config["conv2_out_channels"],
121.     fc1_out_features=model_config["fc1_out_features"],
122.     num_classes=model_config["num_classes"]
123. ).to(device)

```

```

124. criterion = nn.CrossEntropyLoss()
125. learning_rate = 0.001
126. optimizer = optim.Adam(model.parameters(), lr=learning_rate)
127. if 'xpu' in str(device):
128.     try:
129.         print("尝试使用 IPEX.optimize() 进行优化...")
130.         model, optimizer = ipex.optimize(model, optimizer=optimizer,
dtype=torch.float32)
131.         print("IPEX.optimize() 应用成功。")
132.     except Exception as e:
133.         print(f"IPEX.optimize() 失败: {e}. 将不使用 IPEX 优化。")
134.
135. # --- 辅助函数: 获取所有预测和真实标签 ---
136. def get_all_preds_labels(model_to_eval, loader, device_to_use):
137.     model_to_eval.eval() #确保模型在评估模式
138.     all_preds = []
139.     all_labels = []
140.     with torch.no_grad():
141.         for images, labels in loader:
142.             images = images.to(device_to_use)
143.             labels = labels.to(device_to_use)
144.             outputs = model_to_eval(images)
145.             _, predicted = torch.max(outputs.data, 1)
146.             all_preds.extend(predicted.cpu().numpy()) # 收集预测结果
147.             all_labels.extend(labels.cpu().numpy()) # 收集真实标签
148.     return np.array(all_preds), np.array(all_labels)
149.
150. # --- 辅助函数: 绘制混淆矩阵 ---
151. def plot_confusion_matrix_func(y_true, y_pred, classes, title='混淆矩阵'):
152.     cm = confusion_matrix(y_true, y_pred)
153.     disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=classes)
154.     fig, ax = plt.subplots(figsize=(8, 6)) #可以调整 figsize 来改变图像大小
155.     disp.plot(ax=ax, cmap=plt.cm.Blues)
156.     ax.set_title(title)
157.     # 解决中文显示问题
158.     plt.rcParams['font.sans-serif'] = ['SimHei'] # 指定默认字体为黑体
159.     plt.rcParams['axes.unicode_minus'] = False # 解决保存图像是负号 '-' 显示为方块的问题
160.     plt.show()
161.
162. # --- 辅助函数: 绘制性能曲线 ---
163. def plot_performance_curves_func(epoch_history, loss_history, acc_history,
title_prefix=''):
164.     plt.figure(figsize=(12, 4))

```

```

165. plt.rcParams['font.sans-serif'] = ['SimHei']
166. plt.rcParams['axes.unicode_minus'] = False
167. plt.subplot(1, 2, 1)
168. plt.plot(epoch_history, loss_history, label='训练损失 (Training Loss)')
169. plt.xlabel('周期 (Epoch)')
170. plt.ylabel('损失 (Loss)')
171. plt.title(f'{title_prefix}训练损失曲线')
172. plt.legend()
173. plt.subplot(1, 2, 2)
174. plt.plot(epoch_history, acc_history, label='训练准确率 (Training Accuracy)')
175. plt.xlabel('周期 (Epoch)')
176. plt.ylabel('准确率 (Accuracy)')
177. plt.title(f'{title_prefix}训练准确率曲线')
178. plt.legend()
179. plt.tight_layout()
180. plt.show()
181.
182. # --- 5. 训练模型 ---
183. num_epochs = 30
184. print(f"\n 开始在设备 {device} 上训练模型, 共 {num_epochs} 个周期")
185. # 用于存储每个周期的训练损失和准确率
186. epoch_history_list = []
187. train_loss_history = []
188. train_acc_history = []
189. start_train_time = time.time() # 训练开始时间
190. for epoch in range(num_epochs):
191.     model.train()
192.     running_loss = 0.0
193.     correct_train_epoch = 0 # 当前 epoch 的正确数
194.     total_train_epoch = 0 # 当前 epoch 的总数
195.     for i, (images, labels) in enumerate(train_loader):
196.         images = images.to(device)
197.         labels = labels.to(device)
198.         outputs = model(images)
199.         loss = criterion(outputs, labels)
200.         optimizer.zero_grad()
201.         loss.backward()
202.         optimizer.step()
203.         running_loss += loss.item() * images.size(0)
204.         _, predicted = torch.max(outputs.data, 1)
205.         total_train_epoch += labels.size(0)
206.         correct_train_epoch += (predicted == labels).sum().item()
207.         if (i + 1) % 200 == 0:

```

```

208.         print(f'周期 [{epoch+1}/{num_epochs}], 批次 [{i+1}/{len(train_loader)}],
,
209.         f'当前批次损失: {loss.item():.4f}'))
210.     epoch_loss = running_loss / total_train_epoch
211.     epoch_acc = 100 * correct_train_epoch / total_train_epoch
212.     epoch_history_list.append(epoch + 1)
213.     train_loss_history.append(epoch_loss)
214.     train_acc_history.append(epoch_acc)
215.     print(f'周期结束 [{epoch+1}/{num_epochs}], 平均损失: {epoch_loss:.4f}, 训练准确率:
{epoch_acc:.2f}%')
216. end_train_time = time.time() # 训练结束时间
217. total_training_duration = end_train_time - start_train_time
218. print(f"模型训练完成。总训练耗时: {total_training_duration:.2f} 秒")
219. # --- 绘制训练性能曲线 ---
220. plot_performance_curves_func(epoch_history_list, train_loss_history,
train_acc_history, title_prefix='CNN ')
221. # --- 评估训练集整体性能并绘制混淆矩阵 ---
222. print("\n--- 在整个训练集上评估最终模型性能 ---")
223. train_preds, train_labels_true = get_all_preds_labels(model, train_eval_loader,
device)
224. final_train_accuracy = accuracy_score(train_labels_true, train_preds) * 100
225. print(f"在训练集上得到的最终准确率为: {final_train_accuracy:.2f}%")
226. class_names = [f"数字 {i}" for i in range(10)]
227. plot_confusion_matrix_func(train_labels_true, train_preds, classes=class_names,
title='训练集混淆矩阵 (CNN)')
228.
229. # --- 6. 评估模型 (在测试集上) ---
230. print("\n--- 在测试集上评估模型 ---")
231. test_preds, test_labels_true = get_all_preds_labels(model, test_loader, device) # 使
用辅助函数
232. final_test_accuracy = accuracy_score(test_labels_true, test_preds) * 100
233. # 计算测试集上的平均损失
234. print(f"在测试集上得到的准确率为: {final_test_accuracy:.2f}%")
235. plot_confusion_matrix_func(test_labels_true, test_preds, classes=class_names,
title='测试集混淆矩阵 (CNN)')
236. print("\nCNN 模型评估完成。")
237.

```

五、实验结果的分析

1. 卷积神经网络的有效性:

CNN 的卓越性能源于其核心组件: 卷积层能够自动学习图像中的空间层级特征 (从低级的边缘、角点到高级的形状组合), 而池化层则有助于降低特征维度、提取主要特征并引入一定程度的平移不变性。这种结构特别适合处理图像这类具有空间结构的数据。

与实验一中准确率在 55%-84% 区间的朴素贝叶斯分类器相比, CNN 通过深度学习和特征的自动提取, 在准确率上实现了质的飞跃, 更适合此类复杂的模式识别任务。

2. IPEX/XPU 加速效果分析:

实验日志显示 IPEX 成功启用并将计算设备设置为 XPU (Intel Arc 140V GPU (16GB)), 且 `ipex.optimize()` 应用成功。这为在集成显卡上进行有效的深度学习训练提供了支持。

在 Intel Arc 140V GPU (16GB) 的加速下, 30 个周期的训练在不到 10 分钟内完成。训练期间, GPU 利用率保持在较高水平, 有效地分担了计算密集型任务, 缩短了训练时间, 提升了实验效率。

3. 潜在的改进方向:

进一步提升性能: 虽然 99.42% 的准确率已经很高, 但仍可尝试如数据增强 (随机旋转、缩放、平移等)、更复杂的 CNN 架构 (如加入 Batch Normalization 层以加速收敛和提高稳定性, 或使用更深的网络)、学习率衰减策略、Dropout 正则化等方法来探索性能极限或进一步增强模型的泛化能力。

详细错误分析: 可以通过生成混淆矩阵来具体分析模型在哪些数字之间更容易混淆, 从而针对性地改进模型或数据处理。

模型轻量化与部署: 若考虑实际应用, 还可以研究模型剪枝、量化等技术, 以减小模型体积和计算量, 便于部署到资源受限的设备。

总结:

本实验成功构建并训练了一个卷积神经网络用于 MNIST 手写数字识别, 在 Intel 集成显卡上借助 IPEX 实现了高效训练, 并取得了优异的识别准确率。实验结果充分验证了 CNN 在图像分类任务中的强大能力。
